

polars-bio w trybie rozproszonym: Ballista i Sail

Podsumowanie techniczne prac — praca magisterska

Miłosz Kowalewski

26 sierpnia 2026

Spis treści

1. Cel pracy i wymagania promotora	1
2. Architektura docelowa	2
Ballista — integracja bezpośrednia (jedna rodzina DataFusion)	2
Sail — pętla przez UDTF (inna rodzina: Spark Connect)	2
3. Przebieg prac (Fazy 0-C)	3
Faza 0 — porządki	3
Faza A — Ballista, wykonalność lokalna i rozproszona	3
Faza A.5 — COITrees w pełni rozproszonym overlap (opisana szerzej w sekcji 4.1) . .	3
Faza B — Sail, wykonalność lokalna	3
Faza C — rozszerzenie na pozostałe operacje	4
4. Sesja bieżąca: dogłębna weryfikacja wykonalności pełnej dystrybucji	5
4.1 Część 1 — COITrees w pełni rozproszonym overlap na Ballistrze	5
4.2 Część 2 — próba osiągnięcia prawdziwej dystrybucji Saila	6
5. Stan względem pierwotnych wymagań promotora	7
6. Znajdź i różnice semantyczne między silnikami/bibliotekami	8
7. Ograniczenia środowiskowe	8
8. Otwarte kroki i dalsze fazy	8

1. Cel pracy i wymagania promotora

Temat pracy magisterskiej: porównanie udostępnianych na otwartych licencjach rozproszonych silników zapytań pod kątem analiz danych genomicznych, na przykładzie integracji biblioteki **polars-bio** (współautor: promotor, dr inż. Marek Wiewiórka) z dwoma silnikami: **Apache Ballista** oraz **LakeSail/Sail**.

Na podstawie manualnego transkryptu rozmów z promotorem (marzec-sierpień 2026) ustalono następujące, potwierdzone wymagania:

1. Cel: ogólny mechanizm dodawania funkcji użytkownika (UDF/UDTF/UDA) na poziomie DataFusion, wpinany do silnika rozproszonego jako *runtime extension*, **bez forkowania** samego silnika.
2. Docelowo porównanie **dwóch** silników — Ballista i Sail — z pełną implementacją w obu.

3. Aspekt naukowy pracy to wydajna implementacja rozproszona, dobór algorytmów oraz benchmarki.
4. Docelowo zmiany mają trafić do samego polars-bio (nowa reguła optymalizatora: lokalna vs. rozproszona ścieżka wykonania) — nie mają pozostać zbiorem niezależnych skryptów obok biblioteki. Ten krok potraktowano jako opcjonalny i odłożony w czasie (Faza F, patrz niżej), ponieważ wymaga osobnej decyzji promotora jako współautora polars-bio.
5. Zmiana **planu zapytania** (repartycja danych według chromosomu) jest konieczna w obu silnikach; sama serializacja danych nie stanowi problemu, ponieważ oba silniki operują na wspólnym standardzie Apache Arrow.
6. UDF/UDTF ma faktycznie wywoływać silnik polars-bio, a nie reimplementować algorytm od zera.
7. Punkty odniesienia SeQuiLa i Comet odrzucono jako nieadekwatne.

Praca prowadzona jest w cyklu: implementacja kroku → test poprawności względem wyroczeni referencyjnej (`pb.overlap()` i pozostałe funkcje polars-bio uruchomione lokalnie) → poprawki → ponowny test.

2. Architektura docelowa

Z punktu widzenia użytkownika, punktem wejścia pozostaje koncepcyjnie polars-bio (`pb.overlap(df_a, df_b)`), a silnik rozproszony jest wybieranym pod spodem backendem. Ponieważ Ballista i Sail należą do różnych rodzin technologicznych, zastosowano dwa różne wzorce integracji.

Ważne ograniczenie zakresu przyjęte na obecnym etapie: przez całą fazę weryfikacji wykonalności (Fazy 0-D) **nie modyfikuje się kodu źródłowego polars-bio**. Cała logika wyboru silnika żyje jako osobny moduł w repozytorium pracy magisterskiej i woła wyłącznie publiczne, niezmiennicze funkcje z zainstalowanego pakietu polars-bio. Właściwa integracja wewnątrz biblioteki to osobny, opcjonalny krok (Faza F).

Ballista — integracja bezpośrednia (jedna rodzina DataFusion)

```
distributed_overlap(df_a, df_b, engine="ballista")    [kod projektu, poza polars-bio]
-> buduje plan zapytania z operatorem overlap() z datafusion-bio-function-ranges
-> wysyła plan do klastra Ballista (scheduler + executors)
-> scheduler repartycjonuje dane wg chromosomu
-> każdy executor liczy lokalny overlap (COITrees) na swojej partycji
-> wyniki wracają, złożone w jeden DataFrame
```

Sail — pętla przez UDTF (inna rodzina: Spark Connect)

```
distributed_overlap(df_a, df_b, engine="sail")    [kod projektu, poza polars-bio]
-> łączy się z klastrem Sail przez protokół Spark Connect
-> Sail repartycjonuje dane wg chromosomu (standardowy groupBy)
-> zarejestrowany UDTF, wywoływany na każdej partycji, WOŁA Z POWROTEM pb.overlap()
    (niezmieniona, publiczna funkcja z zainstalowanego pakietu polars-bio)
-> wynik wraca przez Saila do kodu projektu -> do użytkownika
```

Dla Saila „UDTF woła polars-bio” i „kod projektu woła Saila” to dwa końce tej samej pętli, a nie konkurencyjne podejścia — i żadne z nich nie modyfikuje źródeł polars-bio.

3. Przebieg prac (Fazy 0-C)

Faza 0 — porządki

Zaktualizowano `analiza_literatury.md` i `architektura_draft.md`, usuwając nieaktualny wniosek „Sail odrzucony” (oparty na wcześniejszym, błędnym założeniu) i opisując aktualny stan: Sail jako kandydat do integracji przez UDTF, z mechanizmem `SailExtension/FFI` dla UDF/optimizer rules dopiero projektowanym po stronie zespołu Sail (dyskusja [sail#2001](#)), niezaimplementowanym w chwili pisania pracy. Zespół SedonaDB prowadzi analogiczną integrację (spatial join, koncepcyjnie bliską genomicznemu overlap) na forku `james-willis/sail` — potraktowane jako materiał referencyjny/literaturowy, potwierdzający że natywna integracja na poziomie planu zapytania w Sailu obecnie wymaga forka. Bez forka Sail oferuje dziś jedynie UDTF (PR #1519 w pysail) — i to przyjęto jako w pełni legalny, docelowy (nie tymczasowy) wynik dla Saila w tej pracy.

Faza A — Ballista, wykonalność lokalna i rozproszona

Katalog roboczy: `ballista_genomics/`.

Krok 1-2. Kluczowe odkrycie na starcie: prawdziwym silnikiem algorytmicznym za polars-bio jest zewnętrzny crate `datafusion-bio-function-ranges` (github.com/biodatageeks/datafusion-bio-functions, Apache-2.0) — eksponuje operacje (`overlap`, `merge`, `nearest`, `coverage`, `subtract`, `cluster`, `complement`) jako gotowe funkcje tabelowe `DataFusion`, z wyborem algorytmu (`COITrees`, `IntervalTree`, `Lapper`, `SuperIntervals`). Wcześniejszy prototyp implementował naiwny predykat `overlap` $O(n \cdot m)$ w UDF-ie, bez użycia tego silnika w ogóle. Dodano crate jako zależność (wymagało to podniesienia wersji `datafusion/ballista` do 53.0.0, bo crate pina tę wersję) i zweryfikowano lokalnie (bez Ballisty) — wynik: 8/8 par identycznych z `pb.overlap()`, tym razem faktycznie liczonych przez `COITrees`.

Krok 3. Przepisano `main.rs` na `SessionContext::standalone_with_state()` (klaster `inproc`: `scheduler` + `executor`). Zapytanie z `overlap(...)` zakończyło się oczekiwanym błędem `LogicalExtensionCodec is not provided` — `DataFusion/Ballista` nie potrafi domyślnie zserializować niestandardowego węzła planu logicznego, żeby przesłać go od klienta do schedulera.

Krok 4 — główny cel Fazy A. Zaimplementowano własny `LogicalExtensionCodec` (`DistOverlapProvider`, delegujący do prawdziwego `OverlapProvider`; kodek serializuje jedynie parametry konstruktora, bez protobuf, ok. 50 linii). Wynik: **8/8 par identycznych z baseline, na prawdziwym klastrze Ballista** (`scheduler` + `executor`, rzeczywisty shuffle sieciowy widoczny w planie jako `ShuffleReaderExec`). Mechanizm wstrzyknięcia kodeka okazał się prostszy niż przewidywano: wystarczy `SessionConfigExt::with_ballista_logical_extension_codec(...)` na `SessionConfig` przed zbudowaniem sesji (re-eksportowane z `ballista::prelude`) — nie trzeba omijać `standalone_with_state()` ani dodawać osobnych zależności `ballista-core/-scheduler/-executor`. Ten etap miał jednak istotny kompromis: plan fizyczny na `executorze` spadał do zwykłego `HashJoinExec` zamiast korzystać z `COITrees` (patrz Faza A.5 niżej — kompromis ten został później naprawiony).

Faza A.5 — COITrees w pełni rozproszonym overlap (opisana szerzej w sekcji 4.1)

Faza B — Sail, wykonalność lokalna

Plik: `sail_overlap_udtf.py`. Zastąpiono `applyInPandas` prawdziwym UDTF (PR #1519 w pysail): klasa z metodą `eval()`, rejestrowana przez `spark.udtf.register(...)`, wywoływana przez

LATERAL overlap_udtf(...). Wewnątrz eval() nadal wołane jest niezmienione pb.overlap().
Wynik: **8/8 par, identyczne z lokalnym** pb.overlap().

Po drodze natrafiono na dwa istotne ograniczenia pysail 0.5.3:

1. **Brak wsparcia dla argumentów typu TABLE w UDTF** — wypróbowano trzy warianty składni (TABLE(...) PARTITION BY w SQL, to samo przez DataFrame API/TableArg, argument TABLE bez opcji) — każdy kończy się innym błędem parsera/silnika. PR #1519 dodał rejestrację UDTF i argumenty **skalarne**, ale nie TABLE.
2. **Błąd: LATERAL <skalarne UDTF> nad danymi rozłożonymi na więcej niż jedną partycję fizyczną gubi lub duplikuje wiersze** (zweryfikowane: 2 grupy na 2 partycjach → jedna zgubiona całkowicie, druga zdublowana dwukrotnie). Wymuszenie jednej partycji (.repartition(1)) przed wywołaniem UDTF naprawia poprawność w 100%, kosztem realnej równoległości na tym etapie — do uwzględnienia przy przyszłych benchmarkach.

Finalne, działające podejście: `groupBy("chrom").agg(collect_list(struct(...))) → .repartition(1)` (workaround na błąd 2) → `LATERAL overlap_udtf(chrom, rows_a, rows_b)`.

Faza C — rozszerzenie na pozostałe operacje

Sprawdzono kod źródłowy wszystkich pozostałych operacji w datafusion-bio-function-ranges: w przeciwieństwie do OverlapProvider (deleguje przez session.sql()), wszystkie pozostałe (MergeProvider, NearestProvider, SubtractProvider, CountOverlapsProvider/coverage, ComplementProvider, ClusterProvider) budują własne, **prywatne**, niestandardowe struktury *Exec bezpośrednio w scan(). Oznacza to, że żadna z tych operacji nie ma tej samej „furtki”, która pozwoliła osiągnąć pełną dystrybucję overlap w Ballistrze — wymagałoby to PhysicalExtensionCodec dla prywatnych typów (współpraca z autorami crate’a albo reimplementacja od zera). Zdecydowano: dla Fazy C testujemy tylko poziom lokalny w Ballistrze oraz Sail (UDTF) — pełna dystrybucja w Ballistrze pozostaje osiągnięciem specyficznym dla overlap (do czasu Fazy A.5, patrz niżej).

Status końcowy wszystkich pięciu operacji z pierwotnego zakresu (overlap, merge, nearest, coverage, subtract) — każda zweryfikowana na obu silnikach względem wyroczni pb.*():

Operacja	Ballista (lokalnie)	Ballista (rozproszone)	Sail (UDTF)	Znalezisko
overlap	TAK:	TAK: (COITrees, Faza A.5)	TAK:	—
merge	TAK:	lokalnie tylko	TAK:	—
nearest	TAK:	lokalnie tylko	TAK:	inna reguła rozstrzygania remisów niż pb.nearest()
coverage	TAK:	lokalnie tylko	TAK:	odwrócona konwencja argumentów względem pb.coverage()
subtract	TAK:	lokalnie tylko	TAK:	—

Pełny pakiet testów: tests/test_*_correctness.py (5 plików), łącznie z test_oracle_sanity.py i test_ballista_overlap.py — 18 testów przechodzących.

Poza pierwotnym zakresem pięciu operacji z transkryptu pozostają cluster i complement, również dostępne w `datafusion-bio-function-ranges` — do ustalenia z promotorem, czy wchodzą w zakres pracy.

4. Sesja bieżąca: dogłębna weryfikacja wykonalności pełnej dystrybucji

Po zakończeniu Fazy C promotor/użytkownik zadał wprost trzy pytania weryfikujące: czy wszystkie elementy planu są zrealizowane, czy da się w rozproszonej Ballistrze użyć COITrees, i czy Sail naprawdę działa w sposób rozproszony z serializowanym planem zapytania. W odpowiedzi przeprowadzono dwa niezależne dochodzenia opisane niżej.

4.1 Część 1 — COITrees w pełni rozproszonym overlap na Ballistrze

Problem. Węzeł planu fizycznego odpowiedzialny za wykonanie overlap przez COITrees to `IntervalJoinExec`, tworzony przez `IntervalJoinPhysicalOptimizationRule` z `datafusion-bio-function-ranges`. Ma on publiczny konstruktor `try_new()` i większość getterów, ale wymaga argumentu typu `ColIntervals` — struktury zdefiniowanej w module, który **nie jest publiczny** (mod `intervals` zamiast `pub mod intervals`). W Rust oznacza to, że typ jest nieosiągalny spoza crate'a, mimo że sama struktura ma widoczność `pub`.

Rozwiązanie. Zamiast rezygnować lub reimplementować algorytm od zera, crate `datafusion-bio-function-ranges` został **zvendorowany lokalnie** (`ballista_genomics/vendor/datafusion-bio-function-ranges/`) z jednoliniową łatką widoczności (`mod intervals` → `pub mod intervals`, plus wygodny re-eksport). Jest to udokumentowane w `vendor/PATCH.md` jako **poprawka w pomocniczej bibliotece, nie fork Ballisty ani Saila** — zmiana czysto kosmetyczna (widoczność modułu), gotowa do zgłoszenia jako mały PR upstream.

Zaimplementowano `IntervalJoinPhysicalCodec: PhysicalExtensionCodec` (`ballista_genomics/src/physical_codec.rs`, ok. 300 linii), który: - serializuje wyrażenia filtra przez istniejące funkcje `datafusion_proto` (`serialize_physical_expr/parse_physical_expr`), zamiast pisać własny format binarny dla wyrażeń, - odtwarza schemat pośredni filtra (`build_filter_schema`) oraz `ColIntervals` (przez zvendorowaną, teraz publiczną funkcję `parse_intervals`) po stronie odbiorcy zamiast serializować je wprost, - koduje ręcznie tylko proste, skończone enumeracje (`JoinSide`, `JoinType`, `PartitionMode`, `Algorithm`) jako pojedyncze bajty.

Napotkany błąd runtime. Po pierwszym udanym uruchomieniu pojawił się błąd "Invalid IntervalSearchJoinExec, unsupported PartitionMode Auto in execute()". Przyczyna: sesja skonfigurowana pod `polars-bio` (`with_config_rt_bio()`) usuwa standardową regułę optymalizatora `join_selection`, która normalnie rozstrzyga `PartitionMode::Auto` na konkretny tryb — bez zastąpienia jej niczym innym. Naprawiono wymuszając w kodeku `Auto` → `Partitioned` podczas deserializacji.

Wynik. 8/8 par, identyczne z baseline, na prawdziwym klastrze Ballista, z `algorithm: Coitrees` widocznym w planie fizycznym executora — pełna dystrybucja z **zachowanym algorytmem COITrees**, nie tylko z generycznym `HashJoinExec`.

Walidacja przenośności do polars-bio. Aby przenieść ten mechanizm z osobnego prototypu badawczego (`ballista_genomics/`) do samego `polars-bio` (Faza F), potrzeba: 1. Zaakceptowania (lub obejścia w inny sposób) jednoliniowej łatki widoczności — najlepiej przez zgłoszenie jej jako PR do `datafusion-bio-function-ranges` i oczekiwanie na wydanie z tą zmianą. 2. Ustandaryzowania miejsca rejestracji `LogicalExtensionCodec/PhysicalExtensionCodec` w momencie tworzenia sesji Ballista przez `polars-bio` (dziś to ręczna konfiguracja `SessionConfig` w prototypie). 3. Rozwiązania problemu `PartitionMode::Auto` w sposób trwalszy niż punktowa łatka w kodeku — docelowo przez przywrócenie/zastąpienie reguły `join_selection`

w `with_config_rt_bio()`. 4. Rozszerzenia tego samego wzorca na pozostałe operacje (`merge/nearest/coverage/subtract`) — wymaga to jednak współpracy z autorami `datafusion-bio-function-ranges` ponieważ ich węzły `*Exec` są prywatne (patrz Faza C wyżej); to większy zakres pracy niż sama łatka widoczności.

Wniosek: przeniesienie do `polars-bio` jest wykonalne technicznie dla `overlap`, ale wymaga świadomej decyzji promotora (jako współautora zarówno `polars-bio`, jak i częściowo `datafusion-bio-function-ranges` w tym ekosystemie) co do akceptacji zależności od zvendorowanej łatki do czasu jej wejścia upstream.

4.2 Część 2 — próba osiągnięcia prawdziwej dystrybucji Saila

Krok 1: analiza trybu `local-cluster`. Sail udostępnia tryb `SAIL_MODE=local-cluster`, sugerujący wieloprocesowe/rozproszone wykonanie z wieloma workerami. Analiza kodu źródłowego Saila (`crates/sail-execution/src/worker_manager/`) wykazała, że istnieją tylko dwie implementacje `WorkerManager`: `LocalWorkerManager` (workery jako aktorzy w jednym procesie, `ActorSystem::spawn`) — używana **zarówno** przez tryb `Local`, jak i `local-cluster` — oraz `KubernetesWorkerManager` (tworzy prawdziwe pody Kubernetes). Genuinie wieloprocesowa/wielomaszynowa dystrybucja wymaga więc trybu `KubernetesCluster`.

Potwierdzono to empirycznie, nie tylko przez czytanie źródeł: zrzut procesów systemowych w trakcie działania klastra `local-cluster` z dwoma workerami (`ps aux + ss -tlnp`) wykazał dokładnie **jeden proces Python** (jeden PID), do którego należały wszystkie cztery nasłuchujące porty — `driver`, `worker 1`, `worker 2` i serwer `Spark Connect`. Brak jakiegokolwiek forka/subprocesu. Dodatkowym, mimowolnym dowodem prawdziwej granicy serializacji (nie tylko wywołania w tym samym wątku) był błąd `TypeError: cannot pickle '_thread.lock' object` przy próbie użycia `threading.Lock()` do synchronizacji dostępu do globalnego kontekstu `DataFusion` w `polars-bio` — `UDTF` jest serializowany przez `cloudpickle` do wysłania tam, gdzie faktycznie się wykonuje, więc obiektu locka nie da się przez tę granicę przenieść. Zastąpiono lock retry-loopem z losowym backoffem — rozwiązanie działa, ale ujawniło przy okazji rzadki (niedeterministyczny) błąd współbieżnego dostępu do globalnego, mutowalnego kontekstu `DataFusion` w samym `polars-bio` przy równoległym wykonaniu wielu partycji na współdzielonym procesie.

Wniosek Części 2, etap 1: tryb `local-cluster` daje jedynie pozór wielowęzłowości (osobne porty, osobne role w logach), ale fizycznie jest jednoprosowy. Prawdziwa dystrybucja wymaga `KubernetesCluster`.

Krok 2: próba `KubernetesCluster` lokalnie (k3s). Ponieważ maszyna testowa (WSL2, 3.5GB RAM łącznie) już raz doprowadziła do zawieszenia całego środowiska przy zwykłej kompilacji Rusta, przed instalacją czegokolwiek zbadano realistyczne wymagania pamięciowe: `k3s` (lżejsza alternatywa niż `kind`) wymaga oficjalnie minimum 2 CPU + 2GB RAM dla pojedynczego węzła serwera — **bez uwzględnienia obciążenia**, i bez wymogu Dockera (własny wbudowany containerd). `kind` nie podaje jasnej minimalnej liczby dla zwykłego uruchomienia, ale architektonicznie jest cięższy (pełne węzły Kubernetesa uruchamiane jako kontenery Dockera).

Jako zabezpieczenie zaplanowano twardy limit pamięci przez `cgroup v2` (`memory.max=1500M`, `memory.swap.max=0`, wymuszający szybki, czysty OOM-kill zamiast powolnego zamulenia całego systemu przez swap) oraz zwiększono ogólny margines systemowy: dodatkowy plik wymiany +2GB i obniżenie `vm.swappiness` z 60 do 10 (kernel wcześniej sięga po reclaim zamiast agresywnego swapowania).

Napotkany problem. Instalacja `k3s` (`INSTALL_K3S_SKIP_START=true`) powiodła się, ale próba uruchomienia go przez `systemd` (`systemctl start k3s`) zakończyła się błędem `System has not been booted with systemd as init system`. Weryfikacja wykazała, że ta instancja WSL **nie ma włączonego systemd** jako PID 1 (`systemctl is-system-running` → `offline`, prawdziwy init to

/init) — wcześniejsza, błędna diagnoza (oparta o samo istnienie binarki systemctl) została skorygowana. Naprawa właściwa (systemd=true w /etc/wsl.conf + restart WSL) została odrzucona jako zbyt ryzykowna — zabiłaby całą trwającą sesję roboczą.

Zamiast tego podjęto próbę uruchomienia k3s server jako zwykłego procesu, ręcznie umieszczonego w ograniczonej cgroupie (bez udziału systemd). Próba zapisu limitów (memory.max, memory.swap.max) do nowo utworzonej cgroupy zakończyła się jednak **odmową dostępu nawet dla roota** (Permission denied) — mimo że kontroler memory widnieje w cgroup.subtree_control rodzica. Przyczyna nie została ostatecznie zdiagnozowana, prawdopodobnie to specyficzna dla tej instancji WSL2 usterka delegacji hierarchii cgroup v2 bez systemd zarządzającego nią centralnie.

Rezultat. Proces k3s server wystartował **bez żadnego nałożonego limitu**. W ciągu ok. 60 sekund, jeszcze przed osiągnięciem pełnej gotowości klastra (trwała jeszcze instalacja domyślnych dodatków: coredns, metrics-server, local-path-provisioner przez klipper-helm), dostępna pamięć systemu spadła z 2,1GB do 171MB. Proces został natychmiast i bezpiecznie zatrzymany (k3s-killall.sh), po czym system wrócił do normy (dostępne 2,0GB). Wykonano pełne sprzątnięcie instalacji (k3s-uninstall.sh).

Wniosek Części 2, etap 2: to nie jest tylko teoretyczne ryzyko wynikające z dokumentacji — to zmierzony, potwierdzony brak wykonalności na tej konkretnej maszynie: nawet zanim k3s zdążył w pełni wystartować, zużył więcej pamięci niż wynosił cały dostępny zapas, a mechanizm bezpiecznego ograniczenia zasobów (cgroup v2) okazał się niedostępny w tym środowisku WSL2 bez systemd.

Ogólny wniosek Części 2: żadna z dwóch dostępnych lokalnie ścieżek do prawdziwej, wieloprocessowej/wielomaszynowej dystrybucji Saila nie jest dziś wykonalna na tej maszynie — local-cluster z powodów architektonicznych samego Saila (jednoprocesowość niezależnie od zasobów), KubernetesCluster/k3s z powodów zasobowych i środowiskowych tej konkretnej instalacji WSL2. Jest to rezultat negatywny, ale rygorystycznie zweryfikowany empirycznie (nie tylko z dokumentacji) po obu stronach — realny i wartościowy punkt porównawczy między dojrzałością architektur dystrybucji obu silników, wart odnotowania w pracy: Ballista osiąga pełną, zweryfikowaną dystrybucję na tej samej maszynie (Część 1), Sail — nie, ze zidentyfikowaną, udokumentowaną przyczyną.

5. Stan względem pierwotnych wymagań promotora

Wymaganie promotora	Status
Mechanizm UDF/UDTF jako runtime extension, bez forka silnika	TAK: Zrealizowane dla obu silników (Ballista: LogicalExtensionCodec/PhysicalExtensionCodec; Sail: UDTF z PR #1519)
Porównanie dwóch silników, pełna implementacja w obu	TAK: Zrealizowane dla wszystkich 5 operacji z pierwotnego zakresu
Wydatna implementacja + dobór algorytmów + benchmarki	CZĘŚCIOWO: Dobór algorytmu (COITrees) potwierdzony w pełni rozproszonej ścieżce Ballisty; benchmarki liczbowe — Faza D, nierozpoczęta
Zmiany docelowo wewnątrz polars-bio	ODŁOŻONE: Świadomie odłożone (Faza F, opcjonalna), wymaga osobnej decyzji promotora
Zmiana planu zapytania (repartycja wg chromosomu) w obu silnikach	TAK: Zweryfikowane w obu (Ballista: shuffle w planie fizycznym; Sail: groupBy("chrom"))

Wymaganie promotora	Status
UDF/UDTF faktycznie woła polars-bio, nie reimplementuje algorytmu	TAK: Potwierdzone w obu (Ballista woła datafusion-bio-function-ranges — silnik pod polars-bio; Sail UDTF woła pb.overlap() wprost)

6. Znaleziska i różnice semantyczne między silnikami/bibliotekami

- **nearest**: natywna implementacja w datafusion-bio-function-ranges i pb.nearest() różnie rozstrzygają remisy (gdy kilku kandydatów ma tę samą, zerową odległość) — obie odpowiedzi poprawne co do dystansu, różny wybór konkretnego partnera.
- **coverage**: pb.coverage(a, b) i natywne SQL-owe coverage('reads', 'targets', ...) mają odwróconą konwencję argumentów — to rzeczywista różnica API między bibliotekami, nie błąd.
- **Sail/pysail 0.5.3**: brak wsparcia dla argumentów TABLE w UDTF; błąd LATERAL nad danymi na wielu partycjach (gubi/duplikuje wiersze, workaround: .repartition(1)); SparkSession. getOrCreate() jako proces-globalny singleton powodujący błędy „Connection refused” po wielokrotnym tworzeniu serwera w jednym procesie (naprawione globalnie przejściem na .create()); @udtf sprawdza tryb (lokalny/zdalny) w momencie definicji klasy, nie rejestracji — klasy trzeba budować przez funkcje fabrykujące wywoływane po utworzeniu sesji.
- **polars-bio**: globalny, mutowalny kontekst DataFusion nie jest bezpieczny przy współbieżnym dostępie z wielu wątków w tym samym procesie (ujawnione przy próbie wielopartycyjnego wykonania UDTF w trybie local-cluster).

7. Ograniczenia środowiskowe

Cała praca prowadzona jest na maszynie WSL2 z **3,5GB RAM łącznie**. Wymusiło to szereg praktyk metodologicznych: kompilacja Rusta wyłącznie z CARGO_BUILD_JOBS=1 (domyślna, równoległa kompilacja raz doprowadziła do zawieszenia całego środowiska), debug = false w profilu kompilacji (unikanie wielogodzinnego narzutu linkera), nieuruchamianie ciężkich procesów Rusta i Pythona równolegle, oraz — jak opisano w sekcji 4.2 — bezpośredni wpływ na wykonalność eksperymentu z Kubernetesem/k3s.

8. Otwarte kroki i dalsze fazy

- **Faza D (benchmarking i GCP)** — nierozpoczęta. Wymaga konteneryzacji obu silników, ustalenia z promotorem dostępu do budżetu/projektu GCP, oraz właściwych pomiarów wydajnościowych przy skalowaniu liczby węzłów i rozmiaru danych.
- **Faza E (synchronizacja z promotorem)** — aktualizacja architektura_draft.md wynikami Faz A-C i tej sesji jako materiał na spotkanie; otwarte pytania z transkryptu: natywny distributed Polars jako trzecia ścieżka porównawcza? Dostępność zasobów GCP? Kwestia zakresu cluster/complement (poza pierwotną piątką operacji).
- **Faza F (zmiany wewnątrz polars-bio, opcjonalna)** — odłożona do czasu potwierdzenia wykonalności w Fazach A-D i osobnej decyzji promotora, zwłaszcza w kontekście zależności od zvendorowanej łatki widoczności opisanej w sekcji 4.1.
- Klaster Ballista jako osobne procesy systemu operacyjnego (nie in-proc) — krok wart wykonania przed Fazą D, żeby przetestować rzeczywistą sieć, nie tylko API in-proc.
- Zgłoszenie łatki widoczności (vendor/PATCH.md) jako PR upstream do biodatageeks/datafusion-bio-function-ranges